GITHUB DEVDAYS

GitHub Copilot

From auto-complete to AI-powered development

SMARTER PROMPTS • FASTER WORKFLOWS • VERIFIED CODE

A practical, secure-by-design developer workshop

01 / THE HOOK

What if your editor could help you think?

The goal is not to type faster. It is to spend more time solving the right problem.

WITHOUT A WORKFLOW

- Search for syntax
- Repeat boilerplate
- Guess at fixes

WITH COPILOT + JUDGMENT

- Describe the outcome
- Generate and iterate
- Test and review

An AI pair programmer — not autopilot

Copilot can assist across the development lifecycle; the developer owns the result.

COMPLETE

Inline suggestions while you type

EXPLAIN

Ask about unfamiliar code

EDIT

Change selected code or files

VERIFY

Generate tests and review changes

A repeatable loop beats a clever prompt

Use this workflow for small functions and multi-file tasks.



Generated code is a proposal — tests and review turn it into an engineering decision.

A prompt is a mini specification

Give Copilot the same clarity you would give a teammate.

CONTEXT	TASK	CONSTRAINTS	ACCEPTANCE	VALIDATION
Where / what code?	What should change?	What must stay true?	How do we define done?	How will we check it?

Weak: “Build a user system.”
Stronger: define scope, constraints, expected behavior, tests, and security requirements.

Ask for an outcome, not a vague miracle

Example: a small, testable Python task.

> **“Implement a JavaScript function that validates a user registration form using the existing HTML structure and CSS styles. Preserve the current interface, use vanilla JavaScript without external libraries, validate the username, email, and password on both empty and invalid inputs, and display clear error messages without reloading the page. Avoid inline event handlers and unnecessary DOM manipulation. Write tests for valid input, missing fields, invalid email, and weak passwords. Explain your assumptions before editing, implement the changes incrementally, and run the tests afterward.”**

Why it works: context + constraints + acceptance criteria + validation.

Ask for an outcome, not a vague miracle

Context

Implement a JavaScript function that validates a user registration form using the existing HTML structure and CSS styles. The form contains username, email, and password fields. Validation should provide clear feedback to users without reloading the page.

Constraints

Use vanilla JavaScript without external libraries. Preserve the existing HTML structure, CSS styles, and user interface. Avoid inline event handlers and unnecessary DOM manipulation. Do not rewrite unrelated code or introduce new dependencies.

Acceptance Criteria

- Valid input is accepted.
- Empty username, email, or password fields display appropriate error messages.
- Invalid email addresses are rejected.
- Weak passwords are rejected according to clearly defined password rules.

Validation

Write and run tests for valid input, missing fields, invalid email addresses, and weak passwords. Test boundary cases where appropriate. Check that error messages appear and clear correctly, the page does not reload, and the existing interface still works. Report the actual test results and fix any failures.

Why it works: context + constraints + acceptance criteria + validation.

Give the model the right information

Context is often more valuable than adding more words.

ASK

- Explain this function
- Find edge cases — don't edit yet
- Suggest the smallest safe fix

PROVIDE

- Relevant files and tests
- Error messages and constraints
- Project conventions

Context syntax and available commands vary by editor and version.

Teach Copilot your project rules

Repository instructions can reduce repeated prompting — but they are not a security boundary.

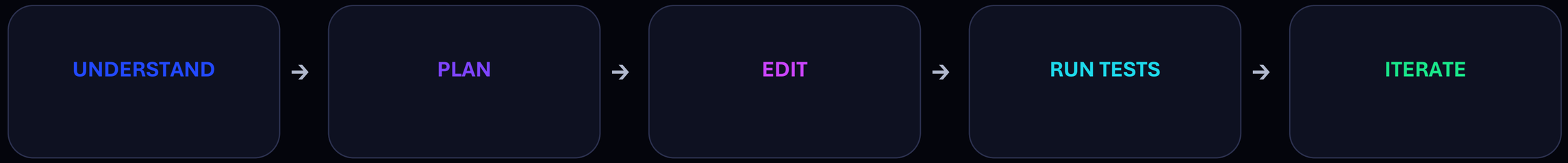
```
# .github/copilot-instructions.md
- Use Python 3.12 and type hints.
- Prefer minimal diffs.
- Never concatenate user input into SQL.
- Use parameterized database queries.
- Add or update tests for behavior changes.
- Never commit real secrets.
- Ask before destructive commands.
```

REMEMBER

- Instructions guide
- Review still required
- Keep rules specific

From answering questions to working through tasks

Where supported, agent workflows can inspect files, edit code, run commands, and iterate.



Use checkpoints. Review terminal commands and every meaningful diff.

One assistant. Many development tasks.

Use Copilot across the lifecycle — while keeping review and ownership with the developer.

EXPLAIN

Understand unfamiliar code

TEST

Cover edge cases

DEBUG

Investigate errors

DOCS

Draft clear documentation

REVIEW

Inspect diffs for risk

SHIP

Summarize changes / PRs

Five prompts worth keeping

Adapt them to your codebase, language, and constraints.

01 Explain this code step by step. State assumptions.

02 List edge cases before changing the implementation.

03 Propose a plan first; do not edit until I approve.

04 Make the smallest safe change and preserve the public interface.

05 Review this diff for correctness, security, and regressions; cite evidence.

Faster development is not blind trust

The developer remains accountable for code, data, and deployed behavior.

REVIEW	Read meaningful changes
TEST	Run relevant checks
PROTECT	Never paste secrets
CONTROL	Approve risky commands
VERIFY	Check docs and assumptions

Copilot can accelerate your work. It cannot take responsibility for your work.

Explore the wider Copilot ecosystem

Advanced capabilities vary by product surface, plan, and release status.

COPILOT CLI

Terminal-based assistance

PLAN MODE

Structure work before edits

CODE REVIEW

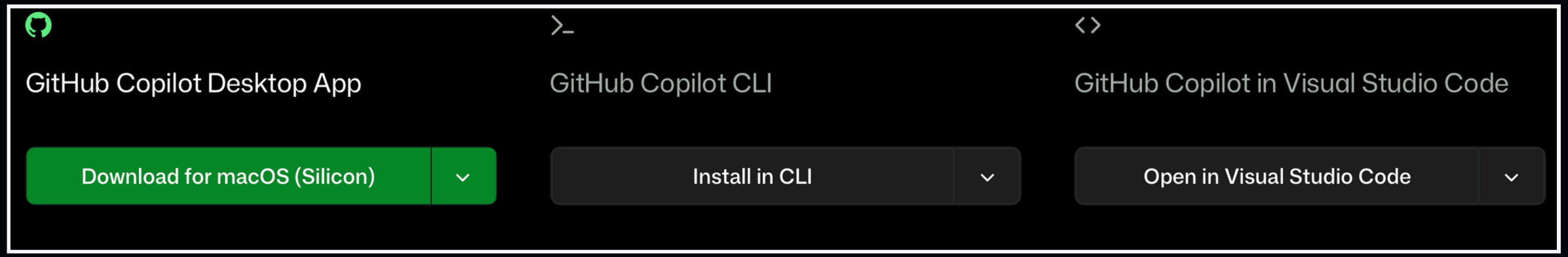
Help inspect changes

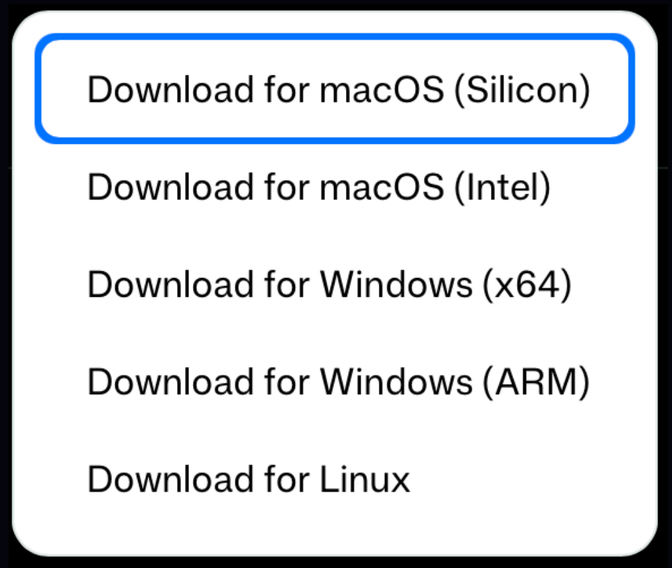
MCP + SKILLS

Extend supported workflows

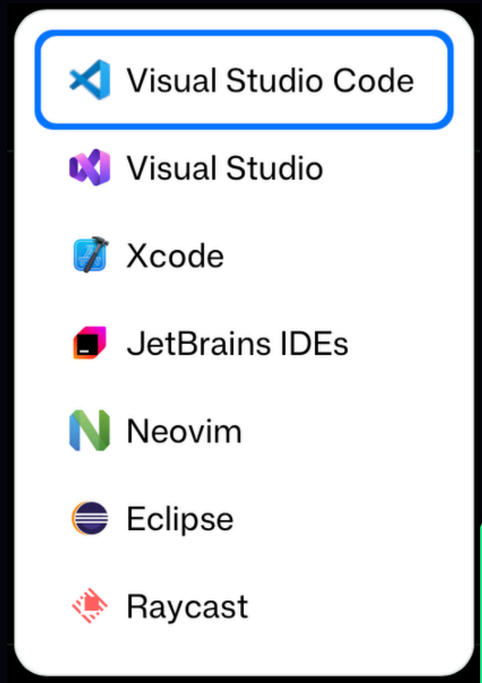
Demo only what you have tested in your own environment.

Copilot


GitHub Copilot Desktop App
Download for macOS (Silicon)
GitHub Copilot CLI
Install in CLI
GitHub Copilot in Visual Studio Code
Open in Visual Studio Code


Download for macOS (Silicon)
Download for macOS (Intel)
Download for Windows (x64)
Download for Windows (ARM)
Download for Linux


Install with script
curl -fsSL https://gh.io/copilot-install | bash
Install with WinGet
winget install GitHub.Copilot
Install with Homebrew
brew install copilot-cli
Install with npm
npm install -g @github/copilot


Visual Studio Code
Visual Studio
Xcode
JetBrains IDEs
Neovim
Eclipse
Raycast

Copilot

Scan



Me...

Your first 20 minutes with Copilot

Start small, then build a habit of verification.

01 **Install the official extension**

02 **Open a small practice repository**

03 **Try inline completion and Chat**

04 **Ask for tests, run them, inspect the diff**

05 **Explore student eligibility and current plan limits**

docs.github.com/en/copilot • education.github.com • skills.github.com

Improve this prompt

Take 20 seconds: what is missing from “Build a login system”?

> **“Build a login system.”**

ADD THE MISSING PIECES

- Framework and existing conventions
- Authentication and password handling
- Tests and failure cases
- Security constraints and validation

A strong prompt defines the problem. A strong developer verifies the solution.

19 / AUDIENCE CHALLENGE

Login System

Task: Implement a secure login system using the existing project's framework, architecture, and coding conventions.

Context: First, inspect the existing codebase and identify its framework, authentication flow, database schema, and current security mechanisms. Explain your assumptions before making changes.

Requirements:

- Authenticate users using their registered email or username and password, consistent with the existing application.
- Use a secure, established password-hashing mechanism; never store or log plaintext passwords.
- Validate inputs on the server side and return clear, non-sensitive error messages.
- Use parameterized database queries or the framework's ORM to prevent SQL injection.
- Manage sessions or authentication tokens securely, following the framework's recommended practices.
- Preserve the existing interface, APIs, and conventions wherever possible.

Constraints:

- Do not introduce unnecessary dependencies or rewrite unrelated code.
- Never hardcode credentials, expose secrets, or reveal whether a specific account exists through distinguishable login errors.
- Apply appropriate protections against brute-force attempts and session-related vulnerabilities using existing project capabilities where available.

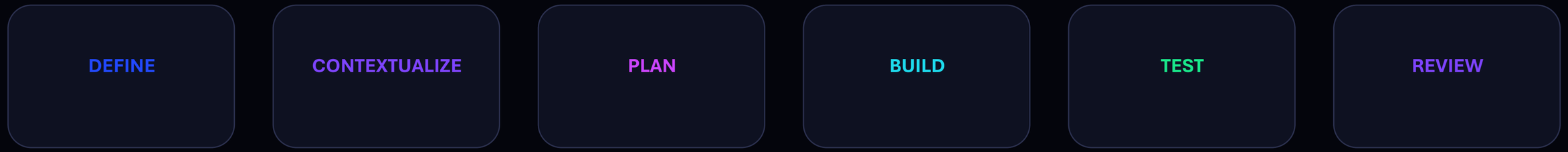
Acceptance criteria:

- Valid credentials authenticate the correct user.
- Incorrect passwords and nonexistent accounts are rejected safely.
- Empty, malformed, and unexpected inputs are handled correctly.
- SQL injection-style input cannot bypass authentication.
- Passwords are verified using the established hashing mechanism.
- Session or token handling follows the framework's security practices.
- Existing functionality remains intact.

Validation:
Write and run automated tests covering successful login, incorrect passwords, nonexistent accounts, invalid inputs, SQL injection attempts, and relevant session behavior. Review the implementation for security weaknesses, fix failures, and report the actual test results. Do not claim tests passed unless they were executed successfully.

Clear prompts. Relevant context. Verified results.

The goal is not to generate more code. It is to make better engineering decisions.



Use AI to accelerate the work — use your judgment to own the result.

Q&A

Don't just code faster. Think better.

Thanks — let's discuss your workflows, questions, and demo ideas.

THANK YOU

Questions?



Sesandu Ramath

sesandu-raj

Computer Science Student with High Interest in Cyber Security and EH.

/Sesandu Ramath

/Sesandu Ramath

/Sesandu Ramath